Adiabatic Quantum Computation of QUBO Problems

Hrishikesh Belagali

CMSE 890 Project

1 Introduction

Combinatorial optimization problems are ubiquitous across fields, including logistics, finance, machine learning, physics, and many others. These problems involve finding an optimal solution to a problem from a finite set of possible solutions, which often grows rapidly with the size of the problem. Many combinatorial optimization problems are NP-hard, meaning that there is no known polynomial-time algorithm to solve them exactly.

Existing methods for solving combinatorial optimization problems include exact algorithms such as branch-and-bound[1], cutting planes[2], dynamic programming[3], and systematic enumeration[4]. However, these methods can be computationally expensive and may not be feasible for large instances. Another class of methods are heuristic algorithms, such as simulated annealing[5], tabu search[6], genetic algorithms[7], and many others. These algorithms are often used when exact methods are too slow, but they do not guarantee optimal solutions.

Quantum computing has been explored as a potential approach to solving combinatorial optimization problems more efficiently than classical algorithms, leveraging phenomena such as superposition and entanglement. One of the earliest algorithms proposed was quantum stochastic optimization, which was a procedure based on the quantum tunnel effect to escape local minima in optimization problems[8]. It was later renamed to quantum annealing (QA) and proposed as an alternative to simulated annealing for solving combinatorial optimization problems, in which quantum fluctuations were used to escape local minima instead of thermal fluctuations[9]. It was later shown that quantum annealing demonstrated faster convergence compared to simulated annealing for certain classes of problems[10]. The experimental implementation of QA in a disordered quantum ferromagnet allowed for the algorithm to be considered as an alternative to the standard gate model, involving a designated quantum device[11].

Adiabatic quantum computation (AQC) is another optimization method which aims to leverage quantum mechanics to solve optimization problems. It relies on the adiabatic theorem of quantum mechanics, and works by encoding the solution to the problem in the ground state of a Hamiltonian. Then the system is slowly evolved from an initial Hamiltonian with a known ground state to the problem Hamiltonian. If the evolution is slow enough, the system will remain in the ground state, yielding the optimal solution to the problem.

It has been shown that non-stoquastic AQC, i.e. AQC with a non-stoquastic Hamiltonian is as powerful as the circuit model of quantum computation, and thus can solve any problem that a universal quantum computer can with atmost polynomial overhead[12].

In this project, we will take a specific class of combinatorial optimization problems, known as Quadratic Unconstrained Binary Optimization (QUBO) problems, and implement an adiabatic evolution algorithm to solve them. We will compare the results with classical heuristic methods and analyze the performance. The goal of the project is to understand the factors that influence AQC's performance and behavior.

2 Methods

2.1 Quadratic Unconstrained Binary Optimization

The Quadratic Unconstrained Binary Optimization (QUBO) problem is a combinatorial optimization problem that minimizes a quadratic objective function defined over binary variables. Formally, given a symmetric matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$, the QUBO problem can be expressed as:

$$\mathbf{x}^* = \arg\min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^\top \mathbf{Q} \mathbf{x}$$

Note that $\mathbf{Q}$ is generally assumed to be symmetric, however this is not a strict requirement since we can always symmetrize it by replacing $\mathbf{Q}$ with $\frac{\mathbf{Q}+\mathbf{Q}^\top}{2}$ without changing the optimization problem. Several combinatorial problems can be reformulated as QUBO problems, including the Max-Cut problem, Graph Coloring, the Traveling Salesman Problem, and many others[13]. In this project, we will formulate the QUBO problem as a Hamiltonian and use adiabatic evolution to find the optimal solution.

2.2 Hamiltonian Formulation

The QUBO problem can be written as

$$f(\mathbf{x}) = \sum_{i=1}^{n} \sum_{j=1}^{n} \mathbf{Q}_{ij} x_i x_j$$

First, we will map the variables from $\{0, 1\}$ to $\{-1, 1\}$ using the transformation

$$x_i = \frac{1 - s_i}{2}$$

where $s_i \in \{-1, 1\}$ is a spin variable. Then, f can be rewritten as

$$\begin{aligned} f(\mathbf{s}) &= \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{Q}_{ij} \frac{1-s_i}{2}\frac{1-s_j}{2} \\ &= \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{J}_{ij} s_i s_j + \sum_{i=1}^{n} \mathbf{h}_i s_i + c \end{aligned}$$

where J_{ij}, h_i, c are derived from $\mathbf{Q}$. The full derivation is given in Appendix **??**. Note that the constant does not affect the optimization, so we can ignore it. The Hamiltonian is then defined by replacing the spin variables with the corresponding Pauli-Z operators.

$$\mathcal{H} = \sum_{i=1}^{n}\sum_{j=1}^{n} \mathbf{J}_{ij} Z_i Z_j + \sum_{i=1}^{n} \mathbf{h}_i Z_i$$

Now, the ground state of $\mathcal{H}$ corresponds to the optimal solution of the QUBO problem. It is found by implementing the adiabatic evolution as a time evolution operator, and measuring the final state in the computational basis.

2.3 Adiabatic Evolution

In adiabatic quantum computation, we start with an initial Hamiltonian $\mathcal{H}_0$ whose ground state is trivial to prepare. Then, we define a time-dependent Hamiltonian that interpolates between $\mathcal{H}_0$ and the problem Hamiltonian $\mathcal{H}$. The solution to the problem is obtained by evolving the system according to the time-dependent Schrodinger equation and measuring the final state. The key constraint is that the evolution must be slow enough to ensure that the system remains in its ground state throughout the process, which is guaranteed by the adiabatic theorem. The total evolution time T must satisfy

$$T \gg \frac{1}{\Delta^2}$$

where Δ is the minimum energy gap between the ground state and the first excited state of the Hamiltonian during the evolution. Here, we define the initial Hamiltonian as

$$\mathcal{H}_0 = -\sum_{i=1}^{n} X_i$$

where X_i is the Pauli-X operator acting on the i-th qubit. The ground state of $\mathcal{H}_0$ is $|+\rangle^{\otimes n}$, which is a uniform superposition of all possible states. Next, we define the problem Hamiltonian $\mathcal{H}_Q$ as derived in the previous section. Finally, we define the time-dependent Hamiltonian $\mathcal{H}_A(t)$ as

$$\mathcal{H}_A(t) = (1 - s(t))\mathcal{H}_0 + s(t)\mathcal{H}_Q$$

where $s(t)$ is a monotonic function that satisfies $s(0) = 0$ and $s(T) = 1$. The evolution is Trotterized**??** into discrete time steps, and at each step, we apply the corresponding unitary operator to evolve the state. Note that as $\Delta t \to 0$, the Hamiltonian is effectively constant during each time step, and the evolution can be approximated by a sequence of unitary operations corresponding to the Hamiltonian at each time step.

2.4 Trotterization

The time evolution operator for a time-dependent Hamiltonian $\mathcal{H}(t)$ is given by the time-ordered exponential:

$$U(T) = \mathcal{T} \exp\left(-i \int_0^T \mathcal{H}(t) dt\right)$$

However, this operator is difficult to compute directly. Instead, we can approximate it using Trotterization, which breaks the evolution into small time steps Δt and approximates the evolution as a product of exponentials of the Hamiltonian at each time step.

Let T be the total evolution time, and let N be the number of time steps, so that $\Delta t = \frac{T}{N}$ by construction. The first order Trotterization is given by

$$e^{-i(A+B)\Delta t} \approx e^{-iA\Delta t} e^{-iB\Delta t}$$

where A and B are the two parts of the Hamiltonian. Higher order Trotterization can be achieved by using more complex sequences of exponentials, such as the second order Trotterization:

$$e^{-i(A+B)\Delta t} \approx e^{-iA\Delta t} e^{-iB\Delta t} e^{-iA\Delta t}$$

The local truncation error is $O(\Delta t^2)$ for the first order Trotterization and $O(\Delta t^3)$ for the second order Trotterization, and the global error is $\mathcal{O}(\frac{T^2}{N})$ and $\mathcal{O}(\frac{T^3}{N^2})$ respectively, assuming that A and B do not have explicit time dependence.

For a time dependent Hamiltonian, we approximate that the Hamiltonian is roughly constant during each time step, and apply the Trotterization to the Hamiltonian at each time t.

2.5 Implementation Details

The implementation of AQC was done using `qiskit`[14] 2.3.1. The Trotterized evolution was implemented using the `PauliEvolutionGate`, `LieTrotter` and `SuzukiTrotter` classes from `qiskit`. The algorithm was tested on the `Aer` simulator, and run on IBM's `ibm_kingston` device to analyze performance in a noisy environment. The `mthree` package was used for readout error mitigation. Calculations were done using `NumPy`[15]. The code is available at this repository.

3 Results

References

[1] An Automatic Method for Solving Discrete Programming Problems | Springer Nature Link. https://link.springer.com/chapter/10.1007/978-3-540-68279-0_5.

[2] On Cutting Planes. In *Annals of Discrete Mathematics*, volume 9, pages 291–296. Elsevier, January 1980.

[3] R. Bellman, R.E. Bellman, and Rand Corporation. *Dynamic Programming*. Rand Corporation Research Study. Princeton University Press, 1957.

[4] Egon Balas. An Additive Algorithm for Solving Linear Programs with Zero-One Variables. *Oper. Res.*, 13(4):517–546, August 1965.

[5] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by Simulated Annealing. *Science*, 220(4598):671–680, May 1983.

[6] Fred Glover and Manuel Laguna. Tabu search. In *Modern Heuristic Techniques for Combinatorial Problems*, pages 70–150. John Wiley & Sons, Inc., USA, May 1993.

[7] Kumara Sastry, David Goldberg, and Graham Kendall. Genetic Algorithms. In Edmund K. Burke and Graham Kendall, editors, *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*, pages 97–125. Springer US, Boston, MA, 2005.

[8] B. Apolloni, C. Carvalho, and D. de Falco. Quantum stochastic optimization. *Stochastic Processes and their Applications*, 33(2):233–244, December 1989.

[9] (PDF) A numerical implementation of quantum annealing. In *ResearchGate*.

[10] Tadashi Kadowaki and Hidetoshi Nishimori. Quantum annealing in the transverse Ising model. *Physical Review E*, 58(5):5355–5363, November 1998.

[11] Quantum Annealing of a Disordered Magnet | Science. https://www.science.org/doi/10.1126/science.284.5415.779.

[12] Adiabatic quantum computation | Rev. Mod. Phys. https://journals.aps.org/rmp/abstract/10.1103/RevModPhys.90.015002.

[13] Fred Glover, Gary Kochenberger, and Yu Du. A Tutorial on Formulating and Using QUBO Models, November 2019.

[14] Ali Javadi-Abhari, Matthew Treinish, Kevin Krsulich, Christopher J. Wood, Jake Lishman, Julien Gacon, Simon Martiel, Paul D. Nation, Lev S. Bishop, Andrew W. Cross, Blake R. Johnson, and Jay M. Gambetta. Quantum computing with Qiskit, June 2024.

[15] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020.